helper, which pointed to an area named Identity. Any subfolder defines an area within the Areas folder by convention. So you have an Identity area; within that, you store the Razor pages in the individual area's own Pages folder. As well as the Login and Register pages, the scaffolder has produced the ViewStart, ViewImports, and a partial for managing validation scripts. All of these override the matching content in the RCL. This is your code now. You own it. Before you start working with the code, you will quickly review its content to better understand two of the principal Identity actors: the `UserManager` and the `SignInManager`.

THE USERMANAGER AND SIGNINMANAGER

The Identity UI is optional. Whether you use it or not, if you build an application based on Identity itself, you will find that you need to customize your authentication workflows to a greater or lesser degree. When you do, you will have to work with the `UserManager` and the `SignInManager`.

The `SignInManager` class provides an API for managing signing a user in. Table 9.6 summarizes the methods you are most likely to use when working with this class.

Table 9.6 Commonly used `SignInManager` methods

Method	Description
`SignInAsync`	Signs the user in and assigns an Identity cookie
`SignOutAsync`	Signs the user out and removes the Identity cookie
`PasswordSignInAsync`	Attempts to sign the user in with the specified password

In addition, the `SignInManager` provides a range of methods that help you work with external login providers, two-factor authentication, lockouts, and so on. When reviewing the `PageModel` code in the Register and Login pages, you can see that the `SignInManager` is injected as a service into the constructors